Proyecto #1

Programación concurrente

Nombre del estudiante:
Joed Francisco Sosa Castellanos

Número de cuenta:
12211012

Sede de estudio:
Campus Tegucigalpa

Docente:
Ing.Roman Pineda

Sección:
CCC415

Fecha de entrega:
15 de agosto de 2026

Índice

1. Introducción y objetivos
2. Entorno de trabajo y estructura del proyecto
3. Paso 1 – Bubble Sort secuencial con contador de intercambios
4. Paso 2 – Dividir el arreglo en dos mitades y mezclar (merge)
5. Paso 3 – Ordenar las dos mitades en hilos (threads) separados
6. Comentarios de implementación y dificultades encontradas
7. Conclusiones generales
8. Guía para demostrar el proyecto paso a paso

Anexo A – Código fuente completo

Anexo B – Salida completa de los experimentos

1. Introducción y objetivos

La programación concurrente agrupa las técnicas que permiten que varias partes de un programa avancen al mismo tiempo. Hoy casi cualquier computadora de escritorio o laptop tiene varios núcleos, así que un programa que corre en un solo hilo de ejecución está desperdiciando la mayor parte del CPU. Para el sistema operativo esto se relaciona directamente con la gestión de procesos e hilos: el planificador (scheduler) decide en qué núcleo y en qué momento corre cada hilo.

En este proyecto se parte de un algoritmo muy sencillo, el **Bubble Sort**, y se va transformando en tres pasos para observar, con datos medidos, qué pasa con el número de intercambios y con el tiempo de ejecución:

- **Paso 1:** Bubble Sort clásico (un solo hilo), modificado para contar los intercambios.
- **Paso 2:** el arreglo se divide en dos mitades, cada mitad se ordena por separado y al final se mezclan (merge). Sigue siendo un programa secuencial.
- **Paso 3:** las dos mitades se ordenan en dos hilos (threads) de Java que corren en paralelo; al terminar ambos se hace un join y un único merge.

Objetivos:

1. Medir cuántos intercambios hace Bubble Sort para 10, 100, 1,000 y 10,000 elementos (y puntos intermedios) y explicar el comportamiento con la teoría.
2. Determinar si dividir el problema en dos mitades mejora el rendimiento y por qué.
3. Usar hilos en Java (Thread, start, join) y comparar el tiempo contra la versión secuencial.

4. Analizar cómo se comportaría la versión con hilos en un CPU single-core y en uno dual-core, simulándolo en la práctica al limitar los núcleos que puede usar el programa.

2. Entorno de trabajo y estructura del proyecto

2.1 Entorno de pruebas

Elemento	Valor
Sistema operativo	Linux (kernel 6.x, 64 bits)
Java	OpenJDK 21.0.10 (javac + java)
Procesador	Intel Xeon @ 2.10 GHz, 4 núcleos
Herramienta para limitar núcleos	taskset (Linux) – en Windows: start /affinity
Datos de prueba	Enteros aleatorios entre 0 y 99,999 con semilla fija (12345 + N)
Medición de tiempo	System.nanoTime(), mediana de 5 repeticiones, con calentamiento previo de la JVM

Tabla 1. Entorno en el que se obtuvieron los resultados de este informe.

Como los datos se generan con una **semilla fija**, los tres programas ordenan exactamente los mismos números para cada tamaño N. Por eso los números de intercambios se pueden comparar directamente entre pasos, y cualquier persona que corra el programa obtiene los mismos intercambios (los tiempos sí cambian según la computadora).

2.2 Estructura del proyecto

```
Proyecto1_SO/
|-- src/
|   |-- bubbleSort.java          Paso 1: bubble sort + contador
|   |-- bubbleSortDividido.java  Paso 2: dos mitades + merge
|   |-- bubbleSortHilos.java     Paso 3: dos hilos + join + merge
|   `-- Comparacion.java         los 3 pasos con los mismos datos -> CSV
|-- resultados/                  salidas de consola (.txt) y tablas (.csv)
`-- informe/                     informe, gráficas y scripts
```

Figura 1. Organización de los archivos del proyecto.

2.3 Compilación y ejecución

Los programas no usan paquetes (package), por lo que se compilan y ejecutan directamente desde la terminal (CMD/PowerShell en Windows o Terminal en Linux/Mac). También se pueden abrir en NetBeans o IntelliJ y ejecutar con el botón Run, pasando los tamaños como argumentos del programa.

```
$ cd Proyecto1_SO/src
$ javac *.java                                  (compila los 4 programas)
$ java bubbleSort                               (Paso 1 - ejemplo de 8 números)
$ java bubbleSort 10 100 1000 10000             (Paso 1 - pruebas)
$ java bubbleSortDividido 10 100 1000 10000     (Paso 2)
$ java bubbleSortHilos 10 100 1000 10000        (Paso 3)
```

```
$ java Comparacion                                (los 3 pasos -> resultados.csv)
```

Figura 2. Comandos para compilar y ejecutar cada paso.

3. Paso 1 – Bubble Sort secuencial con contador de intercambios

3.1 Modificación realizada

Se transcribió el programa `bubbleSort.java` del enunciado. La modificación pedida es contar los intercambios: se agregó una variable `long intercambios` dentro de `bubble_srt`, que se incrementa cada vez que se ejecuta el bloque de intercambio (swap), y el método ahora la devuelve para que `main` la imprima. Se usó `long` y no `int` porque para N grandes el número de intercambios puede pasar de 2,147,483,647.

```
public static long bubble_srt(int a[], int n) {
    int i, j, t = 0;
    long intercambios = 0;                  // <-- NUEVO: contador
    for (i = 0; i < n; i++) {
        for (j = 1; j < (n - i); j++) {
            if (a[j - 1] > a[j]) {
                /* Inicio del intercambio */
                t = a[j - 1];
                a[j - 1] = a[j];
                a[j] = t;
                /* Final del intercambio */
                intercambios++;             // <-- NUEVO: cuenta cada swap
            }
        }
    }
    return intercambios;                    // <-- NUEVO: se devuelve el total
}
```

Figura 3. Método bubble_srt con el contador de intercambios.

Además, el `main` acepta argumentos: si se ejecuta sin argumentos corre el ejemplo original de 8 números; si se le pasan tamaños (por ejemplo `java bubbleSort 10 100 1000 10000`) genera arreglos aleatorios de esos tamaños, los ordena, verifica que quedaron ordenados y reporta intercambios y tiempo.

3.2 Ejecución con el ejemplo del enunciado

```
$ java bubbleSort
Values Before the sort:

12 9 4 99 120 1 3 10
Values after the sort:
1 3 4 9 10 12 99 120
Numero de intercambios: 16
PAUSE
```

Figura 4. Salida del Paso 1 con el arreglo original {12, 9, 4, 99, 120, 1, 3, 10}.

El arreglo de 8 elementos necesitó **16 intercambios**. Ese número no es casualidad: es exactamente la cantidad de **inversiones** del arreglo, es decir, de parejas de números que están en el orden incorrecto (por ejemplo 12 antes que 9). Cada intercambio de Bubble Sort arregla exactamente una inversión, así que el total de intercambios siempre es igual al número de inversiones.

3.3 Pruebas con 10, 100, 1,000 y 10,000 elementos

```
$ java bubbleSort 10 100 1000 10000
Antes:   56022 79543 67812 21265 76510 8557 68389 49959 20302 52275
Despues: 8557 20302 21265 49959 52275 56022 67812 68389 76510 79543
N=    10 | intercambios=          29 |      0.004 ms | ordenado=true
N=   100 | intercambios=       2,450 |      0.143 ms | ordenado=true
N=  1000 | intercambios=     245,941 |      3.786 ms | ordenado=true
N= 10000 | intercambios= 24,998,680 |     74.962 ms | ordenado=true
```

Figura 5. Salida del Paso 1 con 10, 100, 1,000 y 10,000 elementos aleatorios.

3.4 Resultados con puntos intermedios

Para la gráfica se usaron también puntos intermedios (50, 250, 500, 750, 2,500, 5,000 y 7,500). La tabla compara los intercambios medidos con dos referencias: el número de inversiones del arreglo (calculado con otro algoritmo, en `Comparacion.java`) y el valor teórico esperado para datos aleatorios, **n(n−1)/4**.

N	Intercambios medidos	Inversiones	Teórico n(n−1)/4	Medido / teórico	Tiempo (ms)
10	29	29	22	128.9 %	0.001
50	602	602	612	98.3 %	0.003
100	2,450	2,450	2,475	99.0 %	0.011
250	16,817	16,817	15,562	108.1 %	0.077
500	64,804	64,804	62,375	103.9 %	0.278
750	140,719	140,719	140,438	100.2 %	0.527
1,000	245,941	245,941	249,750	98.5 %	0.898
2,500	1,566,239	1,566,239	1,561,875	100.3 %	4.846
5,000	6,259,658	6,259,658	6,248,750	100.2 %	22.861
7,500	14,089,628	14,089,628	14,060,625	100.2 %	58.733
10,000	24,998,680	24,998,680	24,997,500	100.0 %	114.158

Tabla 2. Intercambios y tiempo del Paso 1 (tiempo = mediana de 5 ejecuciones).

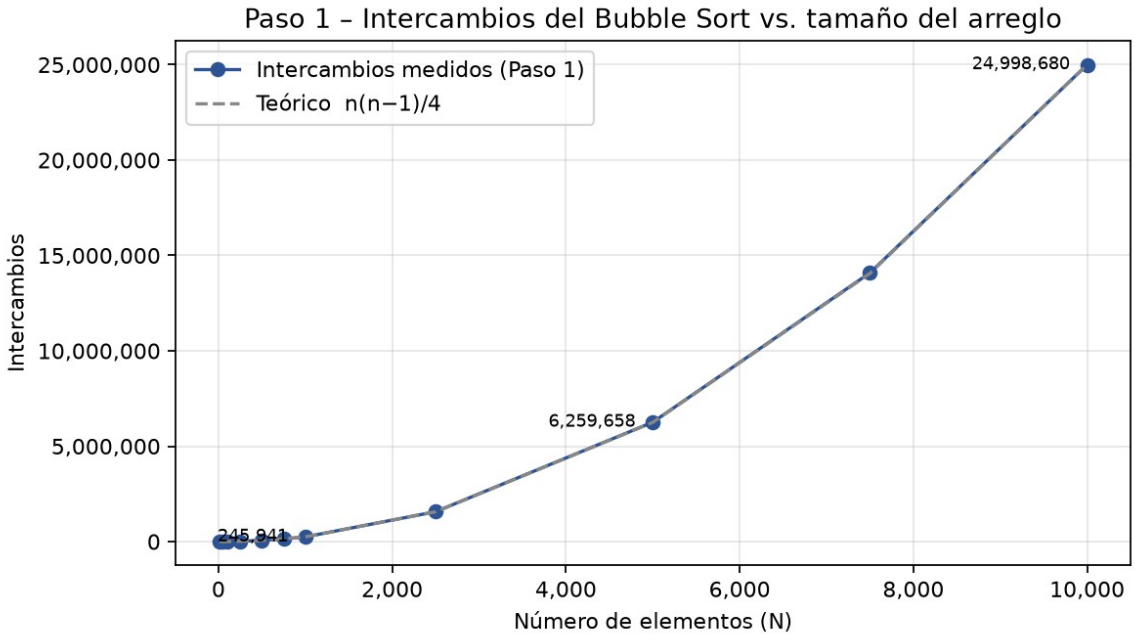


Figura 6. Intercambios del Paso 1 vs. N. La línea punteada es la curva teórica n(n−1)/4.

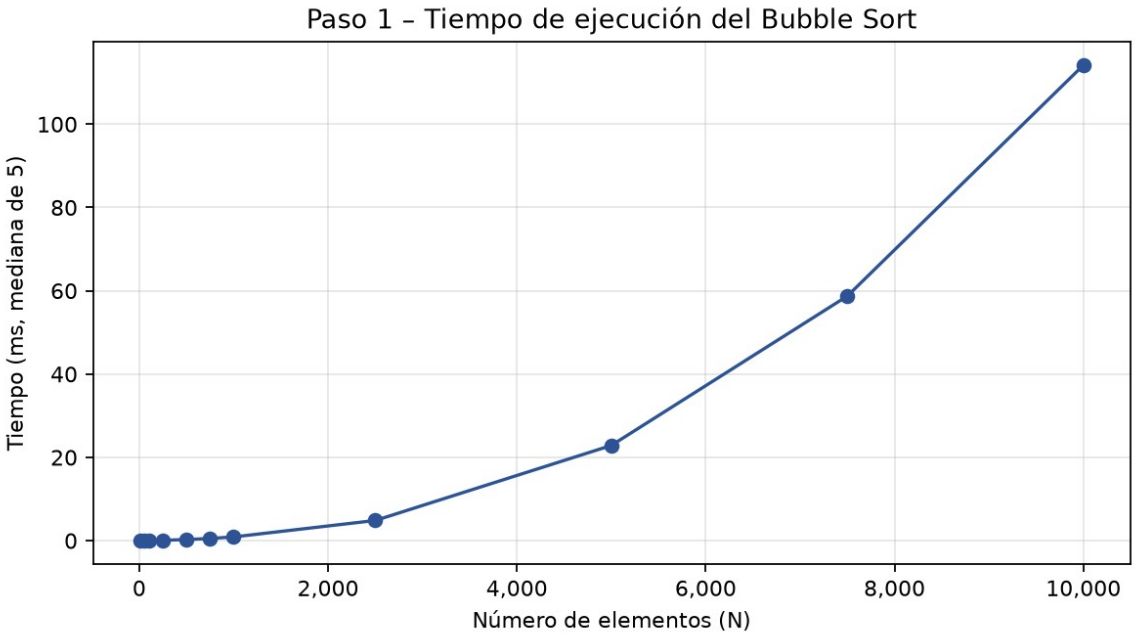


Figura 7. Tiempo de ejecución del Paso 1 vs. N.

3.5 Interpretación de los resultados

La curva es una parábola, no una recta. Al multiplicar N por 10, los intercambios se multiplican aproximadamente por 100: de 10 a 100 elementos pasan de 29 a 2,450; de 1,000 a 10,000 pasan de 245,941 a 24,998,680. Esto confirma que Bubble Sort es un algoritmo **O(N²)**.

¿Por qué exactamente n(n−1)/4? En un arreglo de N elementos existen n(n−1)/2 parejas posibles. Si los datos son aleatorios, cada pareja tiene 50 % de probabilidad de estar invertida, así que en promedio hay n(n−1)/4 inversiones, y como vimos, intercambios = inversiones. La tabla muestra que los valores medidos quedan cerca del teórico: con N muy pequeño la diferencia puede ser grande (con 10 elementos se midieron 29 contra 22.5 esperados, porque depende mucho de cómo salieron esos pocos números), pero para N ≥ 1,000 la diferencia es menor al 2 %, porque con más datos el promedio se estabiliza. La columna de inversiones coincide exactamente con los intercambios en todos los casos.

Comparaciones vs. intercambios. Aunque el número de intercambios depende de los datos, Bubble Sort siempre hace n(n−1)/2 comparaciones (dos veces más que los intercambios promedio). Si el arreglo ya estuviera ordenado haría 0 intercambios pero seguiría haciendo todas las comparaciones; si estuviera en orden inverso haría n(n−1)/2 intercambios (el peor caso).

Tiempos. El tiempo también crece de forma cuadrática: con 1,000 elementos tarda 0.898 ms y con 10,000 tarda 114.158 ms (≈127 veces más para 10 veces más datos). Con 50,000 elementos ya tarda 3.6 segundos. Para N muy pequeños (10–100) los tiempos son de microsegundos y están dominados por ruido del sistema (el planificador, el recolector de basura y el compilador JIT de Java), por lo que no son representativos; por eso cada medición se repitió 5 veces y se tomó la mediana, después de "calentar" la JVM.

4. Paso 2 – Dividir el arreglo en dos mitades y mezclar (merge)

4.1 Descripción del algoritmo

El programa `bubbleSortDividido.java` hace lo siguiente, todo en un solo hilo de ejecución:

1. Divide el arreglo original en `left_array` (primera mitad) y `right_array` (segunda mitad). Si N es impar, la mitad derecha se queda con el elemento sobrante.
2. Llama `bubble_srt(left_array, N/2)` y luego `bubble_srt(right_array, N − N/2)`, una después de la otra.
3. Mezcla (merge) las dos listas ya ordenadas: compara el primer elemento de cada lista, copia el menor al resultado y avanza, hasta terminar ambas. El merge no hace intercambios, solo comparaciones y copias, y cuesta O(N).

```
// 1) Dividir el arreglo en dos partes
int mitad = n / 2;
int left_array[]  = new int[mitad];
```

```
int right_array[] = new int[n - mitad];
System.arraycopy(array, 0,     left_array,  0, mitad);
System.arraycopy(array, mitad, right_array, 0, n - mitad);

// 2) Ordenar cada mitad por separado (secuencial: primero una, luego la otra)
long intIzq = bubble_srt(left_array,  left_array.length);
long intDer = bubble_srt(right_array, right_array.length);

// 3) Mezclar las dos listas ordenadas
int[] resultado = merge(left_array, right_array);

public static int[] merge(int[] left, int[] right) {
    int[] res = new int[left.length + right.length];
    int i = 0, j = 0, k = 0;
    while (i < left.length && j < right.length) {
        if (left[i] <= right[j]) res[k++] = left[i++];
        else                     res[k++] = right[j++];
    }
    while (i < left.length)  res[k++] = left[i++];
    while (j < right.length) res[k++] = right[j++];
    return res;
}
```

Figura 8. Parte central del Paso 2: dividir, ordenar cada mitad y mezclar.

4.2 Ejecución con el ejemplo del enunciado

```
$ java bubbleSortDividido
Antes:          12 9 4 99 120 1 3 10
Mitad izquierda: 4 9 12 99
Mitad derecha:   1 3 10 120
Despues (merge): 1 3 4 9 10 12 99 120
N=     8 | intercambios izq=         3 der=         3 total=         6 |     0.007 ms | ordenado=true
```

Figura 9. Salida del Paso 2 con el arreglo de 8 números: 6 intercambios en lugar de 16.

Con el mismo arreglo de 8 números, ahora solo se hicieron **6 intercambios** (3 en cada mitad) en lugar de 16. Las 10 inversiones restantes eran entre un número de la mitad izquierda y uno de la derecha (por ejemplo 12 y 1); esas "inversiones cruzadas" las resuelve el merge sin hacer ningún intercambio.

4.3 Pruebas con 10, 100, 1,000 y 10,000 elementos

```
$ java bubbleSortDividido 10 100 1000 10000
Antes:          56022 79543 67812 21265 76510 8557 68389 49959 20302 52275
Mitad izquierda: 21265 56022 67812 76510 79543
Mitad derecha:   8557 20302 49959 52275 68389
Despues (merge): 8557 20302 21265 49959 52275 56022 67812 68389 76510 79543
N=    10 | intercambios izq=         5 der=         4 total=         9 |     0.010 ms | ordenado=true
N=   100 | intercambios izq=       584 der=       610 total=     1,194 |     0.067 ms | ordenado=true
N=  1000 | intercambios izq=    62,241 der=    62,046 total=   124,287 |     3.082 ms | ordenado=true
N= 10000 | intercambios izq= 6,350,364 der= 6,186,168 total= 12,536,532 |    29.473 ms | ordenado=true
```

Figura 10. Salida del Paso 2 con 10, 100, 1,000 y 10,000 elementos.

N	Intercambios Paso 1	Intercambios Paso 2	Reducción	ms Paso 1	ms Paso 2	Mejora (P1/P2)
10	29	9	69.0 %	0.001	0.003	0.20x
50	602	262	56.5 %	0.003	0.009	0.33x

N	Intercambios Paso 1	Intercambios Paso 2	Reducción	ms Paso 1	ms Paso 2	Mejora (P1/P2)
100	2,450	1,194	51.3 %	0.011	0.024	0.45x
250	16,817	8,065	52.0 %	0.077	0.066	1.17x
500	64,804	30,051	53.6 %	0.278	0.182	1.53x
750	140,719	70,404	50.0 %	0.527	0.330	1.60x
1,000	245,941	124,287	49.5 %	0.898	0.604	1.49x
2,500	1,566,239	782,377	50.0 %	4.846	2.779	1.74x
5,000	6,259,658	3,092,817	50.6 %	22.861	8.851	2.58x
7,500	14,089,628	7,027,444	50.1 %	58.733	21.825	2.69x
10,000	24,998,680	12,536,532	49.9 %	114.158	47.024	2.43x
20,000	99,458,909	49,823,456	49.9 %	484.351	232.640	2.08x
50,000	622,516,767	311,317,434	50.0 %	3,594.458	1,746.704	2.06x

Tabla 3. Paso 1 vs. Paso 2 con los mismos datos. "Mejora" > 1 significa que el Paso 2 fue más rápido.

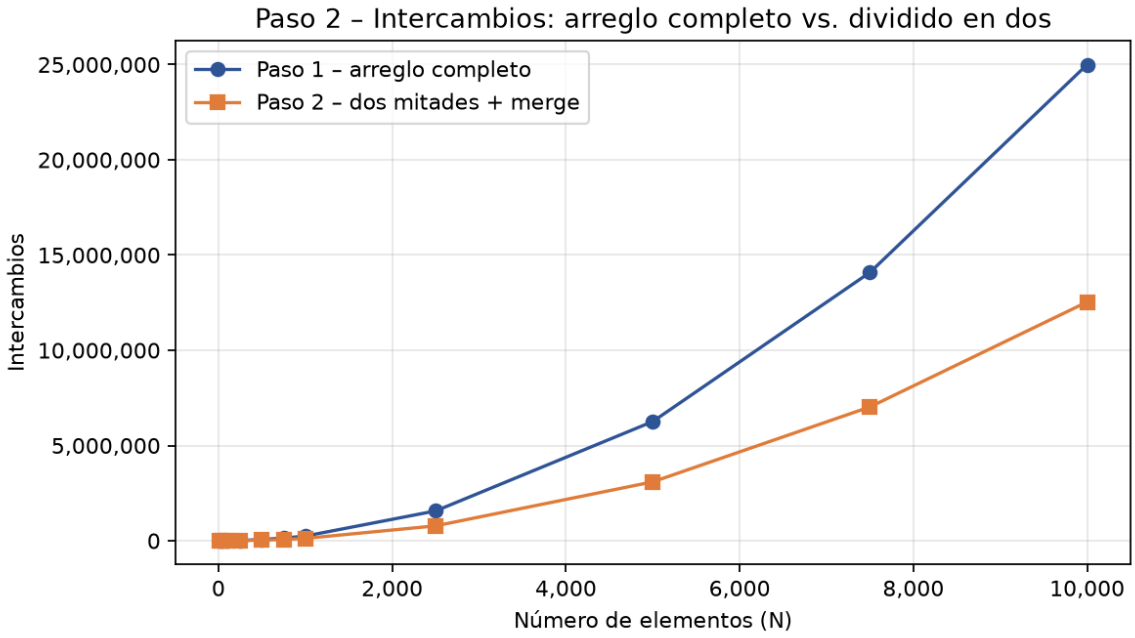


Figura 11. Intercambios del arreglo completo vs. dividido en dos mitades.

4.4 Conclusiones e interpretación del Paso 2

Sí se mejoró el rendimiento, aunque el programa sigue siendo secuencial. Para todos los tamaños los intercambios se redujeron aproximadamente a la **mitad** (≈50 %), y a partir de unos 250 elementos el tiempo también bajó: con 10,000 elementos pasó de 114.158 ms a 47.024 ms y con 50,000 de 3.59 s a 1.75 s. Los principales factores de la mejora son:

- **El costo es cuadrático.** Ordenar N elementos cuesta proporcional a N^2. Ordenar dos mitades cuesta $2 \cdot (N/2)^2 = N^2/2$, o sea la mitad del trabajo. Aplica a comparaciones (de $n(n-1)/2$ a $\approx n^2/4$) y a intercambios (de $\approx n^2/4$ a $\approx n^2/8$).

- **Las inversiones cruzadas se resuelven gratis.** Aproximadamente la mitad de las inversiones son entre un elemento de la izquierda y uno de la derecha. Bubble Sort las resolvería una por una con intercambios; el merge las resuelve con solo N comparaciones.
- **El merge es barato.** Cuesta O(N), que es despreciable frente al $O(N^2)$ del ordenamiento cuando N es grande.
- **Mejor uso de la memoria caché.** Cada mitad ocupa la mitad de memoria, así que cabe mejor en la caché del CPU. Por eso en algunos tamaños (5,000–10,000) la mejora medida fue incluso mayor a 2x; para N muy grandes la mejora se estabiliza cerca de 2x, que es lo que predice la teoría.

Cuándo no conviene: para N pequeños (10–100) el Paso 2 fue igual o más lento, porque crear los dos arreglos, copiar los datos y hacer el merge cuesta más que el poco trabajo que se ahorra. Esta misma idea, aplicada de forma recursiva (dividir cada mitad otra vez, y otra vez…), es exactamente el algoritmo **Merge Sort**, que baja la complejidad hasta O(N log N).

5. Paso 3 – Ordenar las dos mitades en hilos (threads) separados

5.1 Conceptos de hilos en Java

Un **hilo (thread)** es una secuencia de instrucciones que el sistema operativo puede planificar de forma independiente. Todos los hilos de un proceso comparten la misma memoria (los mismos objetos y arreglos), pero cada uno tiene su propia pila y su propio contador de programa. En Java cada objeto `Thread` corresponde a un hilo real del sistema operativo, así que si hay varios núcleos, el planificador del SO puede ejecutar dos hilos al mismo tiempo en núcleos diferentes. Los métodos que se usaron son:

Método	Qué hace
extends Thread / run()	Se crea una clase que hereda de Thread y en run() se pone el código que ejecutará el hilo.
start()	Le pide al sistema operativo crear el hilo y ejecutar run() en paralelo. El hilo main sigue sin esperar.
join()	El hilo que lo llama (main) se bloquea hasta que ese hilo termina. Es el punto de sincronización.
Runtime.availableProcessors()	Indica cuántos núcleos puede usar la JVM.

Tabla 4. Elementos de la API de hilos de Java usados en el Paso 3.

5.2 Implementación

Se creó la clase interna `HiloOrdenador`, que recibe su mitad del arreglo y la ordena dentro de `run()`. Cada hilo guarda sus intercambios en su **propia**

variable, así los dos hilos nunca escriben en la misma memoria y no hace falta sincronización. El hilo principal sigue el modelo **fork–join**:

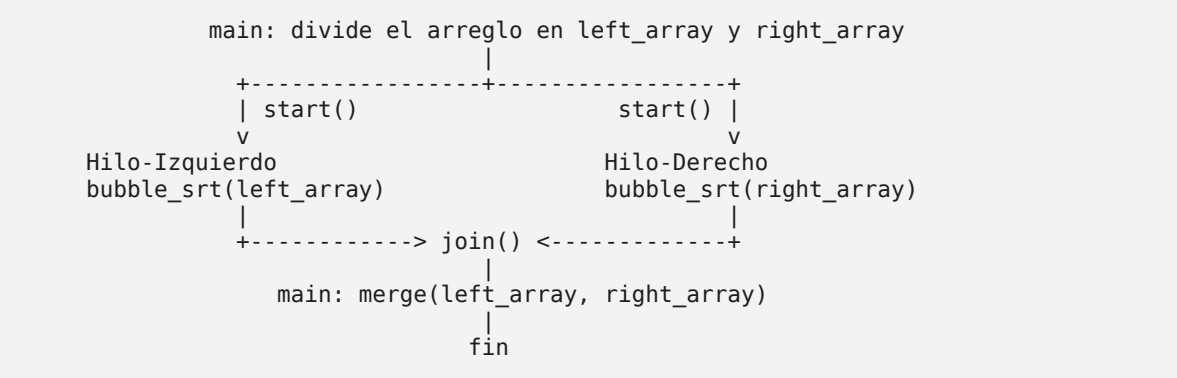


Figura 12. Modelo fork–join del Paso 3: el main crea dos hilos, espera con join() y hace un único merge.

```
static class HiloOrdenador extends Thread {
    private final int[] arreglo;
    long intercambios = 0;            // contador propio de cada hilo

    HiloOrdenador(String nombre, int[] arreglo) {
        super(nombre);
        this.arreglo = arreglo;
    }

    @Override
    public void run() {               // esto corre en el hilo nuevo
        intercambios = bubble_srt(arreglo, arreglo.length);
    }
}

// ----- en el hilo main -----
HiloOrdenador hiloIzq = new HiloOrdenador("Hilo-Izquierdo", left_array);
HiloOrdenador hiloDer = new HiloOrdenador("Hilo-Derecho",   right_array);
hiloIzq.start();                      // ambos hilos empiezan a ordenar en paralelo
hiloDer.start();

hiloIzq.join();                       // main espera a que terminen los dos
hiloDer.join();

int[] resultado = merge(left_array, right_array);   // un único merge al final
```

Figura 13. Clase HiloOrdenador y secuencia start/join/merge del Paso 3.

5.3 Ejecución

```
$ java bubbleSortHilos
Nucleos disponibles para la JVM: 4
Antes:           12 9 4 99 120 1 3 10
Mitad izquierda: 4 9 12 99
Mitad derecha:   1 3 10 120
Despues (merge): 1 3 4 9 10 12 99 120
N=     8 | intercambios izq=         3 der=         3 total=         6 |     0.937 ms | ordenado=true
```

Figura 14. Salida del Paso 3 con el ejemplo de 8 números.

```
$ java bubbleSortHilos 10 100 1000 10000
Nucleos disponibles para la JVM: 4
Antes:           56022 79543 67812 21265 76510 8557 68389 49959 20302 52275
Mitad izquierda: 21265 56022 67812 76510 79543
Mitad derecha:   8557 20302 49959 52275 68389
Despues (merge): 8557 20302 21265 49959 52275 56022 67812 68389 76510 79543
N=    10 | intercambios izq=         5 der=         4 total=           9 |     0.902 ms | ordenado=true
N=   100 | intercambios izq=       584 der=       610 total=       1,194 |     0.555 ms | ordenado=true
N=  1000 | intercambios izq=    62,241 der=    62,046 total=     124,287 |     6.684 ms | ordenado=true
N= 10000 | intercambios izq= 6,350,364 der= 6,186,168 total= 12,536,532 |    17.171 ms | ordenado=true
```

Figura 15. Salida del Paso 3 con 10, 100, 1,000 y 10,000 elementos.

Los intercambios son **idénticos** a los del Paso 2 (los hilos hacen exactamente el mismo trabajo, solo que al mismo tiempo). Lo que cambia es el **tiempo**: con 10,000 elementos el Paso 3 fue claramente más rápido que el Paso 2, pero con 10 y 100 elementos tardó entre 0.5 y 1 ms, mucho más que las versiones secuenciales.

5.4 Comparación de las tres versiones (CPU de 4 núcleos)

N	ms Paso 1	ms Paso 2	ms Paso 3 (hilos)	Paso 2 / Paso 3	Paso 1 / Paso 3
10	0.001	0.003	0.426	0.01x	0.00x
50	0.003	0.009	0.416	0.02x	0.01x
100	0.011	0.024	0.335	0.07x	0.03x
250	0.077	0.066	0.411	0.16x	0.19x
500	0.278	0.182	0.577	0.32x	0.48x
750	0.527	0.330	0.969	0.34x	0.54x
1,000	0.898	0.604	0.829	0.73x	1.08x
2,500	4.846	2.779	2.064	1.35x	2.35x
5,000	22.861	8.851	4.652	1.90x	4.92x
7,500	58.733	21.825	9.298	2.35x	6.32x
10,000	114.158	47.024	17.829	2.64x	6.40x
20,000	484.351	232.640	114.008	2.04x	4.25x
50,000	3,594.458	1,746.704	904.054	1.93x	3.98x

Tabla 5. Tiempos (mediana de 5) de las tres versiones. Valores > 1 en las dos últimas columnas indican que la versión con hilos fue más rápida.

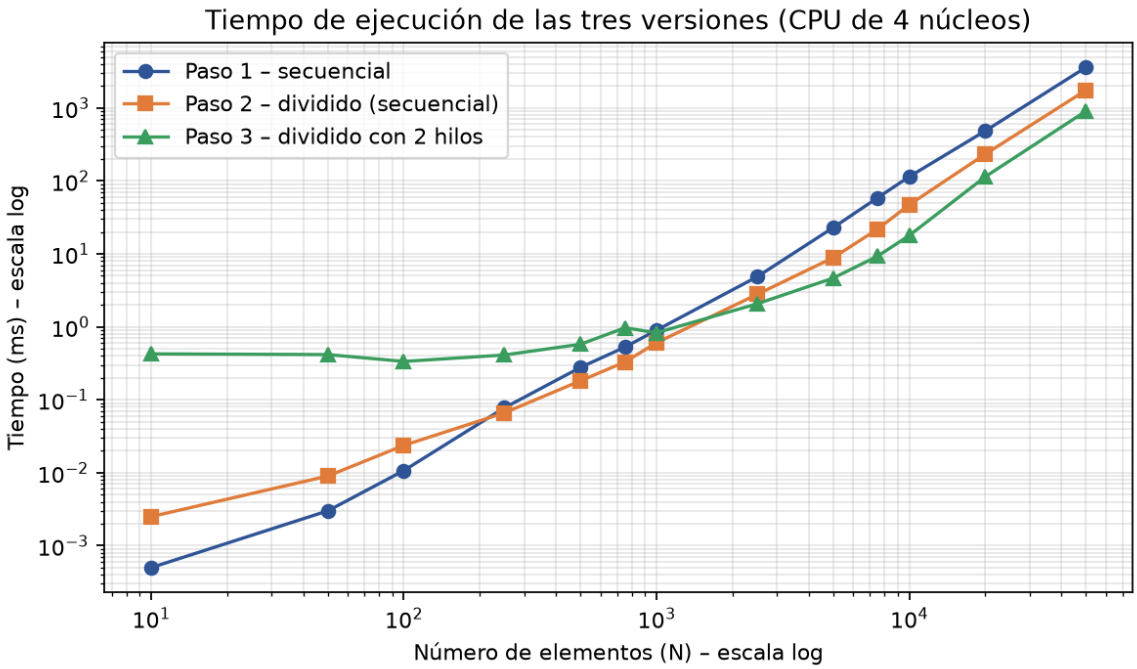


Figura 16. Tiempo de las tres versiones en escala logarítmica.

5.5 ¿Cómo operaría en un CPU single-core o dual-core? (experimento)

Para no quedarse solo en la teoría, se repitió el experimento completo limitando los núcleos que el sistema operativo le permite usar al programa (afinidad de CPU). En Linux se hace con `taskset` y en Windows con `start /affinity`:

```
$ taskset -c 0   java Comparacion resultados_1nucleo.csv 5    (single-core)
$ taskset -c 0,1 java Comparacion resultados_2nucleos.csv 5   (dual-core)
$ java Comparacion resultados_multicore.csv 5                 (4 núcleos)

REM Equivalente en Windows (CMD):
C:\> start /affinity 1 /wait java Comparacion resultados_1nucleo.csv 5
C:\> start /affinity 3 /wait java Comparacion resultados_2nucleos.csv 5
```

Figura 17. Comandos para simular CPUs de 1 y 2 núcleos.

Al correr con `taskset -c 0` el programa confirma que la JVM solo ve un procesador ("Nucleos disponibles para la JVM: 1"). La tabla muestra el speedup del Paso 3 respecto al Paso 2 (tiempo Paso 2 ÷ tiempo Paso 3) en cada caso:

N	1 núcleo ms P2	1 núcleo ms P3	1 núcleo speedup	2 núcleos ms P2	2 núcleos ms P3	2 núcleos speedup	4 núcleos speedup
10	0.002	0.214	0.01x	0.005	0.489	0.01x	0.01x
50	0.005	0.202	0.03x	0.011	0.400	0.03x	0.02x
100	0.015	0.187	0.08x	0.022	0.296	0.07x	0.07x
250	0.052	0.215	0.24x	0.054	0.479	0.11x	0.16x
500	0.135	0.330	0.41x	0.169	0.539	0.31x	0.32x
750	0.263	0.576	0.46x	0.327	0.583	0.56x	0.34x
1,000	0.403	0.907	0.44x	0.526	2.927	0.18x	0.73x

N	1 núcleo ms P2	1 núcleo ms P3	1 núcleo speedup	2 núcleos ms P2	2 núcleos ms P3	2 núcleos speedup	4 núcleos speedup
2,500	2.634	2.518	1.05x	2.397	2.428	0.99x	1.35x
5,000	7.999	10.006	0.80x	9.333	7.081	1.32x	1.90x
7,500	15.370	22.152	0.69x	21.516	13.361	1.61x	2.35x
10,000	29.113	47.264	0.62x	45.481	17.879	2.54x	2.64x
20,000	224.377	246.470	0.91x	227.076	128.404	1.77x	2.04x
50,000	1,701.195	1,830.502	0.93x	1,687.733	851.175	1.98x	1.93x

Tabla 6. Paso 2 (secuencial) vs. Paso 3 (hilos) con 1, 2 y 4 núcleos disponibles.

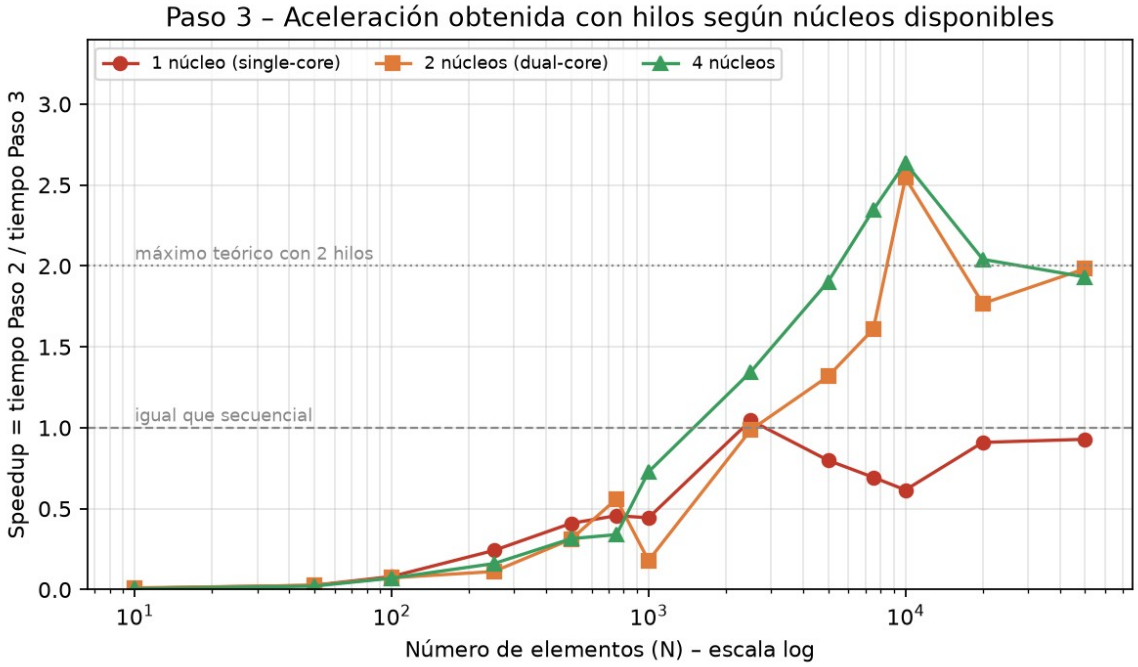


Figura 18. Speedup de la versión con hilos según el número de núcleos. Por encima de 1 los hilos ayudan; por debajo, estorban.

5.6 Conclusiones del Paso 3

Comparando con la evidencia del Paso 2, la respuesta a si ejecutar con hilos es más eficiente, igual o peor es: **depende del tamaño del problema y del número de núcleos**.

- **Multinúcleo y N grande → más eficiente.** Con 2 o más núcleos y N ≥ ~2,500 los hilos son más rápidos. Para N = 50,000 el Paso 3 tardó 904 ms contra 1747 ms del Paso 2 (≈1.9x) y contra 3594 ms del Paso 1 (≈4.0x). Es decir: dividir el problema da ≈2x y paralelizarlo da otro ≈2x.
- **N pequeño → peor.** Crear e iniciar un hilo del sistema operativo cuesta del orden de 0.2–0.4 ms. Con 10–1,000 elementos ordenar tarda menos que eso, así que la versión con hilos llega a ser más de 100 veces más lenta. El paralelismo solo conviene cuando cada hilo tiene suficiente trabajo para compensar el costo de crearlo.
- **Single-core → no mejora (igual o un poco peor).** Con un solo núcleo los dos hilos no pueden correr al mismo tiempo: el planificador del SO los

alterna (time-slicing) con cambios de contexto. El trabajo total es el mismo que en el Paso 2, más el costo de crear hilos y de los cambios de contexto; por eso el speedup medido quedó entre 0.62x y 0.93x para N ≥ 5,000. Aun así, el Paso 3 en un solo núcleo sigue siendo más rápido que el Paso 1, porque conserva la ventaja algorítmica de dividir.

- **Dual-core → hasta ≈2x.** Con dos núcleos cada hilo tiene su propio núcleo y ambas mitades se ordenan realmente al mismo tiempo. Para N = 50,000 el speedup fue 1.98x, muy cerca del máximo teórico de 2x. No llega exactamente a 2 porque dividir, crear hilos y el merge son partes secuenciales (Ley de Amdahl) y porque las mitades pueden tener distinta cantidad de inversiones, entonces un hilo termina antes y el otro marca el tiempo.
- **Más de 2 núcleos no ayuda más.** Con 4 núcleos el speedup fue prácticamente el mismo que con 2, porque el programa solo crea 2 hilos: los otros núcleos quedan sin trabajo. Para aprovecharlos habría que dividir en más partes (por ejemplo, tantas como `availableProcessors()`).
- Los valores mayores a 2x en algunos tamaños intermedios (5,000–10,000) se explican porque cada núcleo tiene su propia caché L1/L2: al repartir las mitades en dos núcleos, cada mitad cabe mejor en caché. También hay variación normal entre mediciones.

5.7 Demostración de una condición de carrera

Durante el desarrollo se probó primero usar un **único contador estático compartido** por ambos hilos. El resultado era incorrecto y cambiaba en cada ejecución. Para demostrarlo se dejó la opción `--compartido`, que además del contador propio de cada hilo incrementa un contador global sin sincronizar:

```
$ java bubbleSortHilos --compartido 1000 10000
Nucleos disponibles para la JVM: 4
N=  1000 | intercambios izq=    62,241 der=    62,046 total=    124,287 |    17.877 ms | ordenado=true
         contador COMPARTIDO = 98,095 (deberia ser 124,287; se perdieron 26,192)
N= 10000 | intercambios izq= 6,350,364 der= 6,186,168 total= 12,536,532 |    58.701 ms | ordenado=true
         contador COMPARTIDO = 8,369,448 (deberia ser 12,536,532; se perdieron 4,167,084)
```

Figura 19. El contador compartido pierde millones de incrementos por la condición de carrera.

La instrucción `contadorCompartido++` parece una sola operación, pero en realidad son tres: leer el valor, sumarle 1 y escribirlo. Si los dos hilos leen el mismo valor al mismo tiempo, los dos escriben el mismo resultado y se pierde un incremento. Esto es una **condición de carrera** sobre una **sección crítica**, el mismo problema que se estudia en la teoría del curso. Las soluciones posibles son `synchronized`, `AtomicLong` o, la que se usó aquí, **no compartir el dato**: cada hilo cuenta en su propia variable y el main suma los dos resultados después del `join()`, lo cual es correcto y además no pierde rendimiento.

6. Comentarios de implementación y dificultades encontradas

- **Código del enunciado con saltos de línea rotos.** Al copiar el código desde el PDF, el texto "Values Before the sort:\n" quedó partido en dos líneas (un String no puede tener un salto de línea real en Java) y `j++` aparecía como `j+` y `+` en líneas separadas. Hubo que reconstruir esas líneas para que compilara.
- **Tamaño impar.** El enunciado sugiere llamar `bubble_srt(right, N/2)` y `bubble_srt(left, N/2)`, pero si N es impar, N/2 redondea hacia abajo y se pierde un elemento. Se resolvió dejando la mitad derecha con tamaño N − N/2.
- **Condición de carrera con el contador** (sección 5.7). Se solucionó con un contador por hilo.
- **start() vs. run().** Si se llama `hilo.run()` en lugar de `hilo.start()`, el código se ejecuta en el hilo main de forma secuencial y no hay paralelismo (el programa funciona, pero no mejora). Es un error fácil de cometer y difícil de notar sin medir el tiempo.
- **Olvidar join().** Sin `join()`, el main hace el merge mientras los hilos todavía están ordenando y el resultado sale desordenado. Por eso cada programa verifica al final que el arreglo quedó ordenado.
- **Medición de tiempos poco confiable.** Las primeras mediciones variaban mucho entre ejecuciones porque Java compila el código "en caliente" (JIT) y el sistema operativo tiene otros procesos. Se resolvió con: `System.nanoTime()` en lugar de `currentTimeMillis()`, una fase de calentamiento, 5 repeticiones con mediana y la misma semilla aleatoria para las tres versiones.
- **Desbordamiento de enteros.** Al calcular el valor teórico n(n−1)/4 con `int`, para N = 50,000 el resultado salió negativo, porque 50,000 × 49,999 ≈ 2,500 millones no cabe en un `int` (máximo ≈ 2,147 millones). Se corrigió usando `long` (`n * (n - 1L) / 4.0`). Por la misma razón los contadores de intercambios son `long`.
- **Simular single-core/dual-core.** No se contaba con máquinas de 1 y 2 núcleos, así que se usó la afinidad de CPU (`taskset` en Linux, `start /affinity` en Windows) para que el sistema operativo solo planifique el proceso en esos núcleos. Se verificó con `Runtime.getRuntime().availableProcessors()`.

7. Conclusiones generales

- Bubble Sort hace en promedio n(n−1)/4 intercambios (uno por cada inversión) y su tiempo crece de forma cuadrática: multiplicar los datos por 10 multiplica el trabajo por ~100.
- Dividir el problema en dos mitades y mezclarlas reduce el trabajo a la mitad aun sin paralelismo, porque $2 \cdot (N/2)^2 = N^2/2$ y el merge resuelve las inversiones cruzadas en tiempo lineal. Es la base de Merge Sort.
- Ejecutar las mitades en hilos separados solo mejora el tiempo si existen núcleos libres para ejecutarlos al mismo tiempo y si el trabajo de cada hilo es mucho mayor que el costo de crearlo. En este experimento, con 2 núcleos se obtuvo casi el máximo teórico de 2x; con 1 núcleo, ninguna mejora.
- Combinando las dos ideas (dividir + paralelizar) la versión final fue ≈4 veces más rápida que el Bubble Sort original para N grandes.
- Los hilos comparten memoria, lo que es cómodo pero peligroso: cualquier dato que dos hilos modifiquen a la vez necesita sincronización o, mejor aún, diseñarse para no compartirse.
- Esto conecta con la gestión de procesos del sistema operativo: los hilos de Java son hilos del SO y es el planificador del sistema operativo quien decide en qué núcleo corre cada uno.

8. Guía para demostrar el proyecto paso a paso

Orden sugerido para presentar el proyecto en vivo (toma unos 5 minutos):

1. Abrir una terminal en la carpeta `src` y compilar: `javac *.java`.
2. **Paso 1:** `java bubbleSort` → mostrar el arreglo antes/después y los 16 intercambios. Luego `java bubbleSort 10 100 1000 10000` → señalar que ×10 datos ≈ ×100 intercambios.
3. **Paso 2:** `java bubbleSortDividido` → mostrar las dos mitades ordenadas, el merge y que ahora son 6 intercambios. Luego `java bubbleSortDividido 10 100 1000 10000` → los intercambios bajan a la mitad.
4. **Paso 3:** `java bubbleSortHilos 10 100 1000 10000` → mismos intercambios que el Paso 2, pero menos tiempo para 10,000 y más tiempo para 10.
5. Abrir el Administrador de tareas (Windows) o `htop` (Linux) y correr `java bubbleSortDividido 50000` y luego `java bubbleSortHilos 50000`: con el Paso 2 se ve un solo núcleo al 100 %, con el Paso 3 se ven dos.
6. **Comparación completa:** `java Comparacion` → tabla con los tres pasos y archivo `resultados.csv` (se abre en Excel para hacer las gráficas).

7. **Single-core:** `taskset -c 0 java Comparacion r1.csv 3` (Linux) o `start /affinity 1 /wait java Comparacion r1.csv 3` (Windows) → los hilos ya no ayudan.
8. **Condición de carrera:** `java bubbleSortHilos --compartido 10000` → el contador compartido pierde incrementos y el valor cambia en cada ejecución.

Anexo A – Código fuente completo

bubbleSort.java (Paso 1)

```
import java.util.Random;

/**
 * PASO 1 - Bubble Sort clasico (secuencial) con contador de intercambios.
 *
 * Uso:
 *   java bubbleSort                   -> ejecuta el ejemplo original de 8 numeros
 *   java bubbleSort 10 100 1000 10000 -> ordena arreglos aleatorios de esos tamanos
 *                                        y reporta intercambios y tiempo
 *
 * Autor: Joed Francisco Sosa Castellanos - 12211012
 */
public class bubbleSort {

    public static void main(String a[]) {
        if (a.length == 0) {
            ejemploOriginal();
        } else {
            for (String arg : a) {
                probarConTamano(Integer.parseInt(arg));
            }
        }
    }

    /** El programa del enunciado, ahora reportando el numero de intercambios. */
    public static void ejemploOriginal() {
        int i;
        int array[] = {12, 9, 4, 99, 120, 1, 3, 10};

        System.out.println("Values Before the sort:\n");
        for (i = 0; i < array.length; i++)
            System.out.print(array[i] + " ");
        System.out.println();

        long intercambios = bubble_srt(array, array.length);

        System.out.print("Values after the sort:\n");
        for (i = 0; i < array.length; i++)
            System.out.print(array[i] + " ");
        System.out.println();
        System.out.println("Numero de intercambios: " + intercambios);
        System.out.println("PAUSE");
    }

    /** Genera un arreglo aleatorio de tamano n, lo ordena y reporta resultados. */
    public static void probarConTamano(int n) {
        int[] array = generarArreglo(n);
        if (n <= 20) imprimir("Antes:   ", array);

        long inicio = System.nanoTime();
        long intercambios = bubble_srt(array, array.length);
        long fin = System.nanoTime();

        if (n <= 20) imprimir("Despues: ", array);
        System.out.printf("N=%6d | intercambios=%,11d | %9.3f ms | ordenado=%s%n",
                n, intercambios, (fin - inicio) / 1e6, estaOrdenado(array));
    }

    /**
     * Bubble sort del enunciado. La unica modificacion es la variable
     * "intercambios", que se incrementa cada vez que se hace un swap.
     */
    public static long bubble_srt(int a[], int n) {
        int i, j, t = 0;
        long intercambios = 0;
        for (i = 0; i < n; i++) {
```

```
            for (j = 1; j < (n - i); j++) {
                if (a[j - 1] > a[j]) {
                    /* Inicio del intercambio */
                    t = a[j - 1];
                    a[j - 1] = a[j];
                    a[j] = t;
                    /* Final del intercambio */
                    intercambios++;
                }
            }
        }
        return intercambios;
    }

    /** Arreglo pseudoaleatorio reproducible: la misma n genera siempre los mismos datos. */
    public static int[] generarArreglo(int n) {
        Random r = new Random(12345L + n);
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = r.nextInt(100000);
        return arr;
    }

    public static boolean estaOrdenado(int[] arr) {
        for (int i = 1; i < arr.length; i++)
            if (arr[i - 1] > arr[i]) return false;
        return true;
    }

    public static void imprimir(String etiqueta, int[] arr) {
        System.out.print(etiqueta);
        for (int x : arr) System.out.print(x + " ");
        System.out.println();
    }
}
```

bubbleSortDividido.java (Paso 2)

```
import java.util.Random;

/**
 * PASO 2 - Bubble Sort dividido en dos mitades + merge (todavia SECUENCIAL).
 *
 * El arreglo original se parte en left_array y right_array, se ordena cada
 * mitad con bubble_srt (una despues de la otra, en el mismo hilo) y al final
 * se mezclan (merge) las dos listas ordenadas en una sola.
 *
 * Uso:
 *   java bubbleSortDividido                   -> ejemplo de 8 numeros del enunciado
 *   java bubbleSortDividido 10 100 1000 10000 -> pruebas con arreglos aleatorios
 *
 * Autor: Joed Francisco Sosa Castellanos - 12211012
 */
public class bubbleSortDividido {

    public static void main(String a[]) {
        if (a.length == 0) {
            int array[] = {12, 9, 4, 99, 120, 1, 3, 10};
            ordenarYReportar(array, true);
        } else {
            for (String arg : a) {
                int n = Integer.parseInt(arg);
                ordenarYReportar(generarArreglo(n), n <= 20);
            }
        }
    }

    public static void ordenarYReportar(int[] array, boolean mostrar) {
        int n = array.length;
```

```
        if (mostrar) imprimir("Antes:          ", array);

        long inicio = System.nanoTime();

        // 1) Dividir el arreglo en dos partes.
        //    Si N es impar, la mitad derecha se queda con el elemento extra.
        int mitad = n / 2;
        int left_array[] = new int[mitad];
        int right_array[] = new int[n - mitad];
        System.arraycopy(array, 0, left_array, 0, mitad);
        System.arraycopy(array, mitad, right_array, 0, n - mitad);

        // 2) Ordenar cada mitad por separado (secuencialmente: primero una, luego la otra)
        long intIzq = bubble_srt(left_array, left_array.length);
        long intDer = bubble_srt(right_array, right_array.length);

        // 3) Mezclar las dos listas ordenadas
        int[] resultado = merge(left_array, right_array);

        long fin = System.nanoTime();

        if (mostrar) {
            imprimir("Mitad izquierda:", left_array);
            imprimir("Mitad derecha:  ", right_array);
            imprimir("Despues (merge):", resultado);
        }
        System.out.printf("N=%6d | intercambios izq=%,10d der=%,10d total=%,11d | %9.3f ms |
ordenado=%s%n",
                n, intIzq, intDer, intIzq + intDer, (fin - inicio) / 1e6,
estaOrdenado(resultado));
    }

    /** Mismo bubble sort del Paso 1: devuelve cuantos intercambios hizo. */
    public static long bubble_srt(int a[], int n) {
        int i, j, t = 0;
        long intercambios = 0;
        for (i = 0; i < n; i++) {
            for (j = 1; j < (n - i); j++) {
                if (a[j - 1] > a[j]) {
                    t = a[j - 1];
                    a[j - 1] = a[j];
                    a[j] = t;
                    intercambios++;
                }
            }
        }
        return intercambios;
    }

    /**
     * Mezcla dos arreglos ordenados en uno solo ordenado.
     * No hace intercambios: solo compara y copia, por eso cuesta O(N).
     */
    public static int[] merge(int[] left, int[] right) {
        int[] res = new int[left.length + right.length];
        int i = 0, j = 0, k = 0;
        while (i < left.length && j < right.length) {
            if (left[i] <= right[j]) res[k++] = left[i++];
            else res[k++] = right[j++];
        }
        while (i < left.length) res[k++] = left[i++];
        while (j < right.length) res[k++] = right[j++];
        return res;
    }

    public static int[] generarArreglo(int n) {
        Random r = new Random(12345L + n); // misma semilla que el Paso 1 -> mismos datos
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = r.nextInt(100000);
        return arr;
    }
```

```
    public static boolean estaOrdenado(int[] arr) {
        for (int i = 1; i < arr.length; i++)
            if (arr[i - 1] > arr[i]) return false;
        return true;
    }

    public static void imprimir(String etiqueta, int[] arr) {
        System.out.print(etiqueta + " ");
        for (int x : arr) System.out.print(x + " ");
        System.out.println();
    }
}
```

bubbleSortHilos.java (Paso 3)

```
import java.util.Random;

/**
 * PASO 3 - Bubble Sort dividido, con cada mitad ordenada en su PROPIO HILO.
 *
 * Hilo main:  divide el arreglo -> crea hilo izquierdo y hilo derecho ->
 *             start() a ambos -> join() espera a que terminen -> merge.
 *
 * Uso:
 *   java bubbleSortHilos                    -> ejemplo de 8 numeros del enunciado
 *   java bubbleSortHilos 10 100 1000 10000 -> pruebas con arreglos aleatorios
 *   java bubbleSortHilos --compartido 10000 -> ademas usa un contador COMPARTIDO
 *                                              sin sincronizar para mostrar la
 *                                              condicion de carrera
 *
 * Autor: Joed Francisco Sosa Castellanos - 12211012
 */
public class bubbleSortHilos {

    /** Contador global compartido por ambos hilos SIN sincronizacion (solo para la demo). */
    static long contadorCompartido = 0;
    static boolean usarCompartido = false;

    /**
     * Hilo que ordena una mitad del arreglo. Cada hilo guarda sus
     * intercambios en su propia variable, asi no hay datos compartidos.
     */
    static class HiloOrdenador extends Thread {
        private final int[] arreglo;
        long intercambios = 0;

        HiloOrdenador(String nombre, int[] arreglo) {
            super(nombre);
            this.arreglo = arreglo;
        }

        @Override
        public void run() {
            intercambios = bubble_srt(arreglo, arreglo.length);
        }
    }

    public static void main(String a[]) throws InterruptedException {
        int idx = 0;
        if (a.length > 0 && a[0].equals("--compartido")) {
            usarCompartido = true;
            idx = 1;
        }
        System.out.println("Nucleos disponibles para la JVM: " +
Runtime.getRuntime().availableProcessors());
        if (idx == a.length) {
            int array[] = {12, 9, 4, 99, 120, 1, 3, 10};
            ordenarYReportar(array, true);
```

```
        } else {
            for (int k = idx; k < a.length; k++) {
                int n = Integer.parseInt(a[k]);
                ordenarYReportar(generarArreglo(n), n <= 20);
            }
        }
    }

    public static void ordenarYReportar(int[] array, boolean mostrar) throws
InterruptedException {
        int n = array.length;
        contadorCompartido = 0;
        if (mostrar) imprimir("Antes:          ", array);

        long inicio = System.nanoTime();

        // 1) Dividir
        int mitad = n / 2;
        int left_array[] = new int[mitad];
        int right_array[] = new int[n - mitad];
        System.arraycopy(array, 0, left_array, 0, mitad);
        System.arraycopy(array, mitad, right_array, 0, n - mitad);

        // 2) Crear un hilo por mitad y arrancarlos: ahora corren en paralelo
        HiloOrdenador hiloIzq = new HiloOrdenador("Hilo-Izquierdo", left_array);
        HiloOrdenador hiloDer = new HiloOrdenador("Hilo-Derecho", right_array);
        hiloIzq.start();
        hiloDer.start();

        // 3) join(): el hilo main se bloquea hasta que ambos terminen
        hiloIzq.join();
        hiloDer.join();

        // 4) Un unico merge al final
        int[] resultado = merge(left_array, right_array);

        long fin = System.nanoTime();

        if (mostrar) {
            imprimir("Mitad izquierda:", left_array);
            imprimir("Mitad derecha:  ", right_array);
            imprimir("Despues (merge):", resultado);
        }
        long total = hiloIzq.intercambios + hiloDer.intercambios;
        System.out.printf("N=%6d | intercambios izq=%,10d der=%,10d total=%,11d | %9.3f ms |
ordenado=%s%n",
                n, hiloIzq.intercambios, hiloDer.intercambios, total, (fin - inicio) / 1e6,
estaOrdenado(resultado));
        if (usarCompartido) {
            System.out.printf("         contador COMPARTIDO = %,d (deberia ser %,d; se
perdieron %,d)%n",
                    contadorCompartido, total, total - contadorCompartido);
        }
    }

    public static long bubble_srt(int a[], int n) {
        int i, j, t = 0;
        long intercambios = 0;
        for (i = 0; i < n; i++) {
            for (j = 1; j < (n - i); j++) {
                if (a[j - 1] > a[j]) {
                    t = a[j - 1];
                    a[j - 1] = a[j];
                    a[j] = t;
                    intercambios++;
                    if (usarCompartido) contadorCompartido++; // carrera: leer-sumar-escribir
no es atomico
                }
            }
        }
        return intercambios;
    }
```

```
    public static int[] merge(int[] left, int[] right) {
        int[] res = new int[left.length + right.length];
        int i = 0, j = 0, k = 0;
        while (i < left.length && j < right.length) {
            if (left[i] <= right[j]) res[k++] = left[i++];
            else res[k++] = right[j++];
        }
        while (i < left.length) res[k++] = left[i++];
        while (j < right.length) res[k++] = right[j++];
        return res;
    }

    public static int[] generarArreglo(int n) {
        Random r = new Random(12345L + n); // misma semilla que Paso 1 y Paso 2
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = r.nextInt(100000);
        return arr;
    }

    public static boolean estaOrdenado(int[] arr) {
        for (int i = 1; i < arr.length; i++)
            if (arr[i - 1] > arr[i]) return false;
        return true;
    }

    public static void imprimir(String etiqueta, int[] arr) {
        System.out.print(etiqueta + " ");
        for (int x : arr) System.out.print(x + " ");
        System.out.println();
    }
}
```

Comparacion.java (Experimento comparativo)

```
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.Arrays;

/**
 * Experimento completo: ejecuta las tres versiones (Paso 1, 2 y 3) sobre
 * EXACTAMENTE los mismos datos, repite varias veces cada medicion, se queda
 * con la mediana del tiempo y guarda todo en un CSV (se puede abrir en Excel
 * para hacer las graficas).
 *
 * Uso:
 *   java Comparacion                          -> tamanos por defecto, 5 repeticiones,
resultados.csv
 *   java Comparacion salida.csv 7 10 100 1000 -> archivo, repeticiones y tamanos propios
 *
 * Autor: Joed Francisco Sosa Castellanos - 12211012
 */
public class Comparacion {

    static final int[] TAMANOS_DEFECTO = {10, 50, 100, 250, 500, 750, 1000, 2500, 5000, 7500,
10000, 20000, 50000};

    public static void main(String[] args) throws Exception {
        String archivo = args.length > 0 ? args[0] : "resultados.csv";
        int repeticiones = args.length > 1 ? Integer.parseInt(args[1]) : 5;
        int[] tamanos = TAMANOS_DEFECTO;
        if (args.length > 2) {
            tamanos = new int[args.length - 2];
            for (int i = 2; i < args.length; i++) tamanos[i - 2] = Integer.parseInt(args[i]);
        }

        System.out.println("Nucleos disponibles para la JVM: " +
```

```
Runtime.getRuntime().availableProcessors());
        System.out.println("Repeticiones por medicion: " + repeticiones + " (se reporta la
mediana)");
        calentarJVM();

        System.out.printf("%n%7s | %12s | %12s | %12s | %12s | %12s | %10s | %10s | %10s | %8s
| %8s%n",
                "N", "inversiones", "n(n-1)/4", "int. Paso1", "int. Paso2", "int. Paso3",
                "ms Paso1", "ms Paso2", "ms Paso3", "P1/P2", "P2/P3");

        try (PrintWriter csv = new PrintWriter(new FileWriter(archivo))) {

csv.println("N,inversiones,teorico_n(n-1)/4,intercambios_paso1,intercambios_paso2,intercambios
_paso3,"
                    +
"ms_paso1,ms_paso2,ms_paso3,speedup_p1_vs_p2,speedup_p2_vs_p3,speedup_p1_vs_p3");
            for (int n : tamanos) {
                int[] original = bubbleSort.generarArreglo(n);
                int[] esperado = original.clone();
                Arrays.sort(esperado);

                long inversiones = contarInversiones(original.clone());
                double teorico = n * (n - 1L) / 4.0;

                long[] r1 = new long[1], r2 = new long[1], r3 = new long[1];
                double[] t1 = new double[repeticiones], t2 = new double[repeticiones], t3 =
new double[repeticiones];
                for (int k = 0; k < repeticiones; k++) {
                    t1[k] = medirPaso1(original, esperado, r1);
                    t2[k] = medirPaso2(original, esperado, r2);
                    t3[k] = medirPaso3(original, esperado, r3);
                }
                double m1 = mediana(t1), m2 = mediana(t2), m3 = mediana(t3);

                System.out.printf("%7d | %,12d | %,12.0f | %,12d | %,12d | %,12d | %10.3f |
%10.3f | %10.3f | %7.2fx | %7.2fx%n",
                        n, inversiones, teorico, r1[0], r2[0], r3[0], m1, m2, m3, m1 / m2,
m2 / m3);
                csv.printf(java.util.Locale.US, "%d,%d,%.1f,%d,%d,%d,%.4f,%.4f,%.4f,%.3f,%.3f,
%.3f%n",
                        n, inversiones, teorico, r1[0], r2[0], r3[0], m1, m2, m3, m1 / m2,
m2 / m3, m1 / m3);
            }
        }
        System.out.println("\nResultados guardados en " + archivo);
    }

    // ---------- Paso 1: bubble sort completo ----------
    static double medirPaso1(int[] original, int[] esperado, long[] intercambios) {
        int[] a = original.clone();
        long ini = System.nanoTime();
        intercambios[0] = bubbleSort.bubble_srt(a, a.length);
        long fin = System.nanoTime();
        verificar(a, esperado, "Paso 1");
        return (fin - ini) / 1e6;
    }

    // ---------- Paso 2: dos mitades en secuencia + merge ----------
    static double medirPaso2(int[] original, int[] esperado, long[] intercambios) {
        int[] a = original.clone();
        long ini = System.nanoTime();
        int mitad = a.length / 2;
        int[] left = Arrays.copyOfRange(a, 0, mitad);
        int[] right = Arrays.copyOfRange(a, mitad, a.length);
        long i1 = bubbleSortDividido.bubble_srt(left, left.length);
        long i2 = bubbleSortDividido.bubble_srt(right, right.length);
        int[] res = bubbleSortDividido.merge(left, right);
        long fin = System.nanoTime();
        intercambios[0] = i1 + i2;
        verificar(res, esperado, "Paso 2");
        return (fin - ini) / 1e6;
    }
```

```
    // ---------- Paso 3: dos mitades en hilos paralelos + join + merge ----------
    static double medirPaso3(int[] original, int[] esperado, long[] intercambios) {
        int[] a = original.clone();
        long ini = System.nanoTime();
        int mitad = a.length / 2;
        int[] left = Arrays.copyOfRange(a, 0, mitad);
        int[] right = Arrays.copyOfRange(a, mitad, a.length);
        bubbleSortHilos.HiloOrdenador h1 = new bubbleSortHilos.HiloOrdenador("Hilo-Izquierdo",
left);
        bubbleSortHilos.HiloOrdenador h2 = new bubbleSortHilos.HiloOrdenador("Hilo-Derecho",
right);
        h1.start();
        h2.start();
        try {
            h1.join();
            h2.join();
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
        int[] res = bubbleSortHilos.merge(left, right);
        long fin = System.nanoTime();
        intercambios[0] = h1.intercambios + h2.intercambios;
        verificar(res, esperado, "Paso 3");
        return (fin - ini) / 1e6;
    }

    /** Ejecuta cada version varias veces para que el compilador JIT optimice antes de medir.
*/
    static void calentarJVM() {
        int[] datos = bubbleSort.generarArreglo(3000);
        int[] esp = datos.clone();
        Arrays.sort(esp);
        long[] dummy = new long[1];
        for (int i = 0; i < 10; i++) {
            medirPaso1(datos, esp, dummy);
            medirPaso2(datos, esp, dummy);
            medirPaso3(datos, esp, dummy);
        }
    }

    static void verificar(int[] obtenido, int[] esperado, String paso) {
        if (!Arrays.equals(obtenido, esperado))
            throw new IllegalStateException(paso + " no ordeno correctamente");
    }

    static double mediana(double[] v) {
        double[] c = v.clone();
        Arrays.sort(c);
        int m = c.length / 2;
        return c.length % 2 == 1 ? c[m] : (c[m - 1] + c[m]) / 2.0;
    }

    /**
     * Cuenta las inversiones (pares i<j con a[i] > a[j]) usando merge sort, O(N log N).
     * Bubble sort hace exactamente un intercambio por cada inversion, asi que
     * este numero debe coincidir con los intercambios del Paso 1.
     */
    static long contarInversiones(int[] a) {
        if (a.length < 2) return 0;
        int mitad = a.length / 2;
        int[] izq = Arrays.copyOfRange(a, 0, mitad);
        int[] der = Arrays.copyOfRange(a, mitad, a.length);
        long inv = contarInversiones(izq) + contarInversiones(der);
        int i = 0, j = 0, k = 0;
        while (i < izq.length && j < der.length) {
            if (izq[i] <= der[j]) a[k++] = izq[i++];
            else {
                a[k++] = der[j++];
                inv += izq.length - i;
            }
        }
```

```
        while (i < izq.length) a[k++] = izq[i++];
        while (j < der.length) a[k++] = der[j++];
        return inv;
    }
}
```

Anexo B – Salida completa de los experimentos

Comparación con 4 núcleos

```
$ java Comparacion resultados/resultados_multicore.csv 5
Nucleos disponibles para la JVM: 4
Repeticiones por medicion: 5 (se reporta la mediana)

      N |  inversiones |     n(n-1)/4 |   int. Paso1 |   int. Paso2 |   int. Paso3 |   ms Paso1 |   ms Paso2 |   ms Paso3 |
P1/P2 |    P2/P3
     10 |           29 |           23 |           29 |            9 |            9 |      0.000 |      0.002 |      0.426 |
0.20x |    0.01x
     50 |          602 |          613 |          602 |          262 |          262 |      0.003 |      0.009 |      0.417 |
0.33x |    0.02x
    100 |        2,450 |        2,475 |        2,450 |        1,194 |        1,194 |      0.011 |      0.024 |      0.335 |
0.45x |    0.07x
    250 |       16,817 |       15,563 |       16,817 |        8,065 |        8,065 |      0.078 |      0.066 |      0.411 |
1.17x |    0.16x
    500 |       64,804 |       62,375 |       64,804 |       30,051 |       30,051 |      0.278 |      0.182 |      0.577 |
1.53x |    0.32x
    750 |      140,719 |      140,438 |      140,719 |       70,404 |       70,404 |      0.527 |      0.330 |      0.969 |
1.60x |    0.34x
   1000 |      245,941 |      249,750 |      245,941 |      124,287 |      124,287 |      0.898 |      0.604 |      0.829 |
1.49x |    0.73x
   2500 |    1,566,239 |    1,561,875 |    1,566,239 |      782,377 |      782,377 |      4.846 |      2.779 |      2.064 |
1.74x |    1.35x
   5000 |    6,259,658 |    6,248,750 |    6,259,658 |    3,092,817 |    3,092,817 |     22.861 |      8.850 |      4.652 |
2.58x |    1.90x
   7500 |   14,089,628 |   14,060,625 |   14,089,628 |    7,027,444 |    7,027,444 |     58.733 |     21.825 |      9.298 |
2.69x |    2.35x
  10000 |   24,998,680 |   24,997,500 |   24,998,680 |   12,536,532 |   12,536,532 |    114.158 |     47.024 |     17.830 |
2.43x |    2.64x
  20000 |   99,458,909 |   99,995,000 |   99,458,909 |   49,823,456 |   49,823,456 |    484.351 |    232.640 |    114.008 |
2.08x |    2.04x
  50000 |  622,516,767 |  624,987,500 |  622,516,767 |  311,317,434 |  311,317,434 |   3594.458 |   1746.703 |    904.054 |
2.06x |    1.93x

Resultados guardados en resultados/resultados_multicore.csv
```

Comparación con 2 núcleos (taskset -c 0,1)

```
$ taskset -c 0,1 java Comparacion resultados/resultados_2nucleos.csv 5
Nucleos disponibles para la JVM: 2
Repeticiones por medicion: 5 (se reporta la mediana)

      N |  inversiones |     n(n-1)/4 |   int. Paso1 |   int. Paso2 |   int. Paso3 |   ms Paso1 |   ms Paso2 |   ms Paso3 |
P1/P2 |    P2/P3
     10 |           29 |           23 |           29 |            9 |            9 |      0.001 |      0.005 |      0.489 |
0.17x |    0.01x
     50 |          602 |          613 |          602 |          262 |          262 |      0.004 |      0.011 |      0.400 |
0.38x |    0.03x
    100 |        2,450 |        2,475 |        2,450 |        1,194 |        1,194 |      0.011 |      0.022 |      0.297 |
0.52x |    0.07x
    250 |       16,817 |       15,563 |       16,817 |        8,065 |        8,065 |      0.099 |      0.054 |      0.479 |
1.84x |    0.11x
    500 |       64,804 |       62,375 |       64,804 |       30,051 |       30,051 |      0.251 |      0.168 |      0.539 |
1.49x |    0.31x
    750 |      140,719 |      140,438 |      140,719 |       70,404 |       70,404 |      0.506 |      0.327 |      0.583 |
1.55x |    0.56x
   1000 |      245,941 |      249,750 |      245,941 |      124,287 |      124,287 |      0.826 |      0.526 |      2.927 |
1.57x |    0.18x
   2500 |    1,566,239 |    1,561,875 |    1,566,239 |      782,377 |      782,377 |      4.474 |      2.397 |      2.428 |
1.87x |    0.99x
   5000 |    6,259,658 |    6,248,750 |    6,259,658 |    3,092,817 |    3,092,817 |     24.849 |      9.333 |      7.081 |
2.66x |    1.32x
   7500 |   14,089,628 |   14,060,625 |   14,089,628 |    7,027,444 |    7,027,444 |     60.752 |     21.516 |     13.361 |
2.82x |    1.61x
  10000 |   24,998,680 |   24,997,500 |   24,998,680 |   12,536,532 |   12,536,532 |    115.704 |     45.481 |     17.879 |
2.54x |    2.54x
  20000 |   99,458,909 |   99,995,000 |   99,458,909 |   49,823,456 |   49,823,456 |    498.071 |    227.076 |    128.404 |
2.19x |    1.77x
  50000 |  622,516,767 |  624,987,500 |  622,516,767 |  311,317,434 |  311,317,434 |   3492.564 |   1687.733 |    851.175 |
2.07x |    1.98x
```

```
Resultados guardados en resultados/resultados_2nucleos.csv
```

Comparación con 1 núcleo (taskset -c 0)

```
$ taskset -c 0 java Comparacion resultados/resultados_1nucleo.csv 5
Nucleos disponibles para la JVM: 1
Repeticiones por medicion: 5 (se reporta la mediana)

      N |  inversiones |     n(n-1)/4 |   int. Paso1 |   int. Paso2 |   int. Paso3 |   ms Paso1 |   ms Paso2 |   ms Paso3 |
P1/P2 |    P2/P3
     10 |           29 |           23 |           29 |            9 |            9 |      0.000 |      0.002 |      0.214 |
0.20x |    0.01x
     50 |          602 |          613 |          602 |          262 |          262 |      0.003 |      0.005 |      0.202 |
0.63x |    0.03x
    100 |        2,450 |        2,475 |        2,450 |        1,194 |        1,194 |      0.016 |      0.015 |      0.187 |
1.06x |    0.08x
    250 |       16,817 |       15,563 |       16,817 |        8,065 |        8,065 |      0.071 |      0.052 |      0.215 |
1.36x |    0.24x
    500 |       64,804 |       62,375 |       64,804 |       30,051 |       30,051 |      0.183 |      0.135 |      0.330 |
1.36x |    0.41x
    750 |      140,719 |      140,438 |      140,719 |       70,404 |       70,404 |      0.378 |      0.263 |      0.576 |
1.44x |    0.46x
   1000 |      245,941 |      249,750 |      245,941 |      124,287 |      124,287 |      0.581 |      0.403 |      0.907 |
1.44x |    0.44x
   2500 |    1,566,239 |    1,561,875 |    1,566,239 |      782,377 |      782,377 |      4.568 |      2.634 |      2.518 |
1.73x |    1.05x
   5000 |    6,259,658 |    6,248,750 |    6,259,658 |    3,092,817 |    3,092,817 |     18.210 |      7.999 |     10.006 |
2.28x |    0.80x
   7500 |   14,089,628 |   14,060,625 |   14,089,628 |    7,027,444 |    7,027,444 |     60.389 |     15.370 |     22.152 |
3.93x |    0.69x
  10000 |   24,998,680 |   24,997,500 |   24,998,680 |   12,536,532 |   12,536,532 |    110.898 |     29.114 |     47.264 |
3.81x |    0.62x
  20000 |   99,458,909 |   99,995,000 |   99,458,909 |   49,823,456 |   49,823,456 |    544.879 |    224.377 |    246.470 |
2.43x |    0.91x
  50000 |  622,516,767 |  624,987,500 |  622,516,767 |  311,317,434 |  311,317,434 |   3626.332 |   1701.195 |   1830.502 |
2.13x |    0.93x

Resultados guardados en resultados/resultados_1nucleo.csv
```